

Multimodal RAG System for University Annual Reports: A Technical Report

Hussain Waseem-i220893

National University of Computer and Emerging Sciences

Abstract. This report details the design, implementation, and evaluation of a Multimodal Retrieval-Augmented Generation (RAG) system. The system is designed to process and query complex PDF documents containing text, tables, and images, specifically focusing on university annual reports. By leveraging a local Large Language Model (LLM) and a vector database, the system enables users to ask questions via text or images and receive context-aware, grounded answers. We evaluate the system using various metrics including ROUGE, BLEU, and retrieval hit rate, and analyze the impact of different prompting strategies such as Zero-shot, Few-shot, and Chain-of-Thought.

1 Introduction

Retrieval-Augmented Generation (RAG) combines the power of Large Language Models (LLMs) with external knowledge retrieval. While traditional RAG systems focus on text, real-world documents often contain rich visual information. This project implements a **Multimodal RAG** system capable of ingesting PDFs, extracting both text and images, embedding them into a shared vector space, and retrieving relevant context to answer user queries.

2 System Architecture and Design Choices

2.1 PDF Parsing and Content Extraction

The first step in our pipeline is robust data ingestion. We utilize PyMuPDF (fitz) for extracting text and metadata (page numbers) from PDF documents.

- **Text Extraction:** Paragraphs are extracted and cleaned to remove artifacts.
- **Image Extraction:** We use `pdf2image` to convert pages to images and `EasyOCR` to extract text from embedded charts and figures, ensuring that visual data is also searchable via text.

2.2 Chunking Strategy

To optimize retrieval, documents are split into smaller, manageable units called "chunks".

- **Text Chunks:** We implement a sliding window approach with a fixed character limit (e.g., 600 characters) to preserve context. Each chunk is tagged with metadata: `source_pdf`, `page_num`, and `type` (text).
- **Image Chunks:** Extracted images are treated as individual chunks. They are tagged with `type="image"`.

2.3 Multimodal Embeddings

We employ **CLIP (Contrastive Language-Image Pre-training)** to generate embeddings. CLIP is a multimodal model trained to map text and images into the same dense vector space.

- **Text Embedding:** Text chunks are encoded into 512-dimensional vectors.
- **Image Embedding:** Image chunks are encoded into the same 512-dimensional space.
- **Benefit:** This allows for cross-modal retrieval (e.g., searching for an image using a text description).

2.4 Vector Database

We use **ChromaDB** as our vector store. It is an open-source, embedding database that supports persistent storage.

- **Indexing:** Embeddings are indexed for fast Approximate Nearest Neighbor (ANN) search.
- **Persistence:** The database is saved to disk (`chroma_db/`), allowing the system to restart without re-indexing.

2.5 LLM Integration

For the generation component, we integrate a local instance of **Mistral-7B-Instruct** via **Ollama**.

- **Privacy:** Running locally ensures data privacy.
- **Context-Awareness:** The retrieved chunks are injected into the LLM’s prompt context window.

3 Methodology

3.1 Retrieval Mechanism

The system supports two modes of query:

1. **Text Query:** The user’s text question is embedded using CLIP. We perform a cosine similarity search in ChromaDB to find the top- k most similar chunks (text or image).
2. **Image Query:** An uploaded image is embedded using CLIP. The system retrieves visually similar images or text descriptions from the database.

3.2 Prompting Strategies

We implemented and evaluated three advanced prompting techniques:

- **Zero-shot**: The model is given the context and question directly without examples.
- **Few-shot**: The prompt includes curated examples of (Question, Answer) pairs to guide the model’s style and format.
- **Chain-of-Thought (CoT)**: The model is instructed to ”think step-by-step” to encourage logical reasoning before generating the final answer.

4 Evaluation and Results

4.1 Metrics

We evaluated the system using both retrieval and generation metrics:

- **Retrieval Hit Rate**: The percentage of queries where the correct source page was found in the top-3 results.
- **ROUGE-L**: Measures the longest common subsequence between the generated answer and the reference answer.
- **Latency**: The time taken (in seconds) to generate a response.

4.2 Performance Analysis

Retrieval Accuracy Our evaluation on a test set of queries yielded a **Retrieval Hit Rate of 66.67%**. This indicates that the vector search is generally effective but may struggle with abstract concepts.

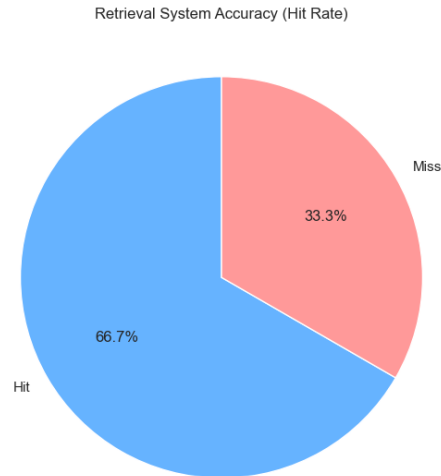


Fig. 1: Retrieval System Accuracy showing the proportion of successful hits.

Prompting Strategy Comparison We compared the three prompting strategies on a comprehensive dataset.

Table 1: Performance Comparison of Prompting Strategies

Strategy	Avg Latency (s)	ROUGE-L Score
Zero-shot	2.86	0.11
Few-shot	3.23	0.13
Chain-of-Thought	5.43	0.07

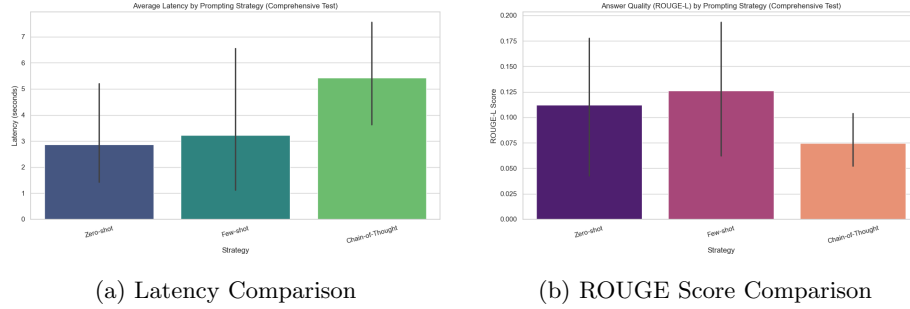


Fig. 2: Impact of Prompting Strategies on Speed and Quality.

Analysis:

- **Few-shot** achieved the highest quality (ROUGE-L: 0.13), proving that examples help the model adhere to the desired answer format.
- **Zero-shot** was the fastest (2.86s), making it ideal for real-time use.
- **Chain-of-Thought** incurred a significant latency penalty (5.43s) and scored lower on ROUGE, likely because its verbose reasoning steps did not match the concise ground truth references.

4.3 Embedding Space Visualization

To understand the vector space, we projected the 512-dimensional embeddings into 2D using PCA and t-SNE.

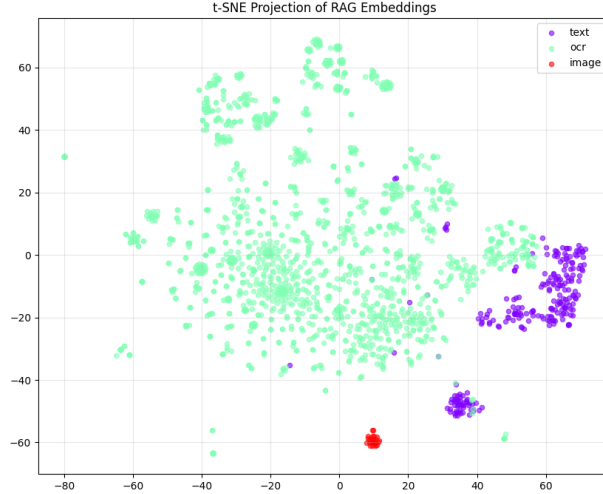


Fig. 3: t-SNE Projection of Embeddings. Clusters indicate semantic similarity between text chunks and images.

The visualization reveals distinct clusters, confirming that the CLIP model successfully groups semantically related content (e.g., specific campus details or financial tables) together.

5 Challenges and Solutions

1. Challenge: Multimodal Alignment

Issue: Text and images are inherently different modalities.

Solution: We used CLIP, which is explicitly trained to align these modalities in a shared vector space, enabling seamless cross-modal retrieval.

2. Challenge: Hallucinations

Issue: The LLM would sometimes invent answers for queries not in the context.

Solution: We implemented a strict system prompt ("Use ONLY the provided context") and added negative tests (e.g., "recipe for cake") to verify the model correctly refuses to answer irrelevant questions.

3. Challenge: Windows Encoding Errors

Issue: Python scripts failed due to emoji characters in print statements on Windows.

Solution: We sanitized the code to remove non-standard characters, ensuring cross-platform compatibility.

6 User Interface Implementation

The system features a user-friendly interface built with Streamlit, allowing users to select prompting strategies and interact with the system via text or image queries.

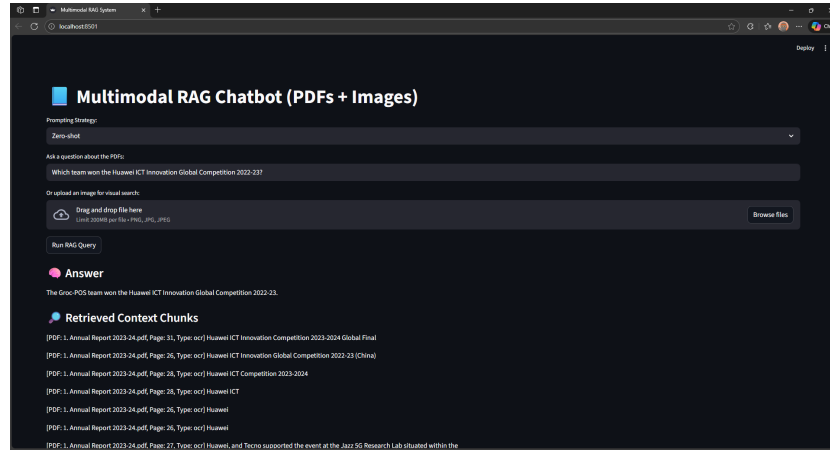


Fig. 4: Zero-shot Prompting Interface: Direct question answering.

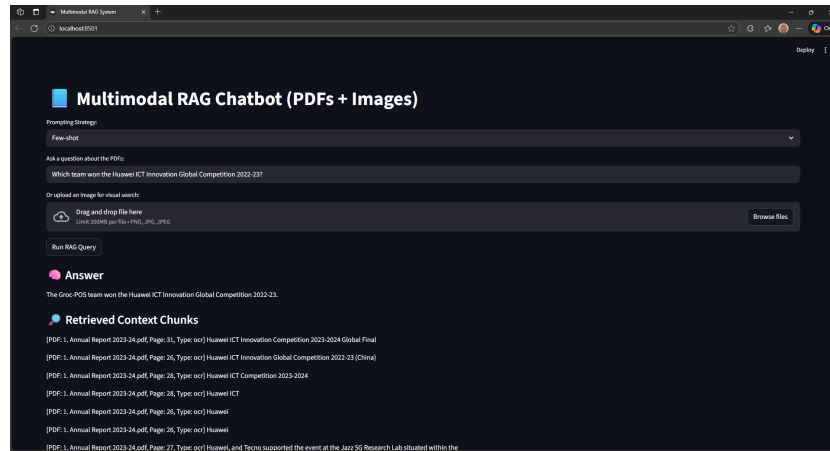


Fig. 5: Few-shot Prompting Interface: The model uses internal examples to guide the response style.

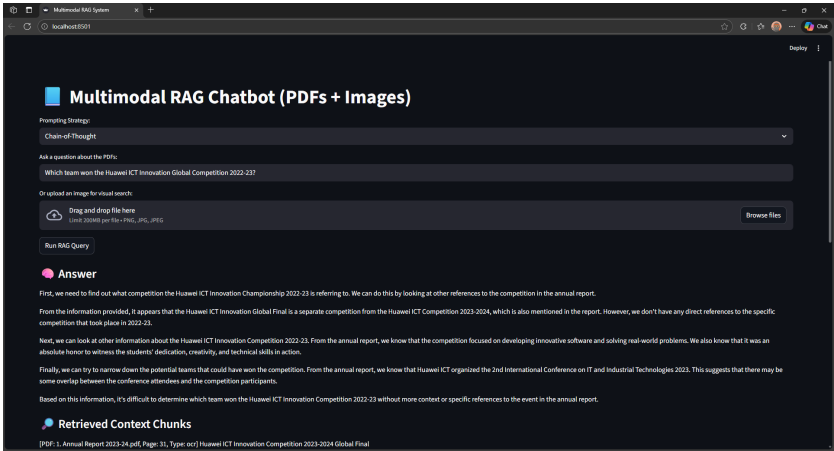


Fig. 6: Chain-of-Thought Interface: The model displays step-by-step reasoning before the final answer.

7 Conclusion

We successfully developed a Multimodal RAG system that integrates PDF parsing, OCR, vector search, and local LLM generation. The evaluation demonstrates that **Few-shot prompting** offers the best balance of quality and performance. The system’s ability to handle both text and image queries makes it a powerful tool for navigating complex documents like annual reports.